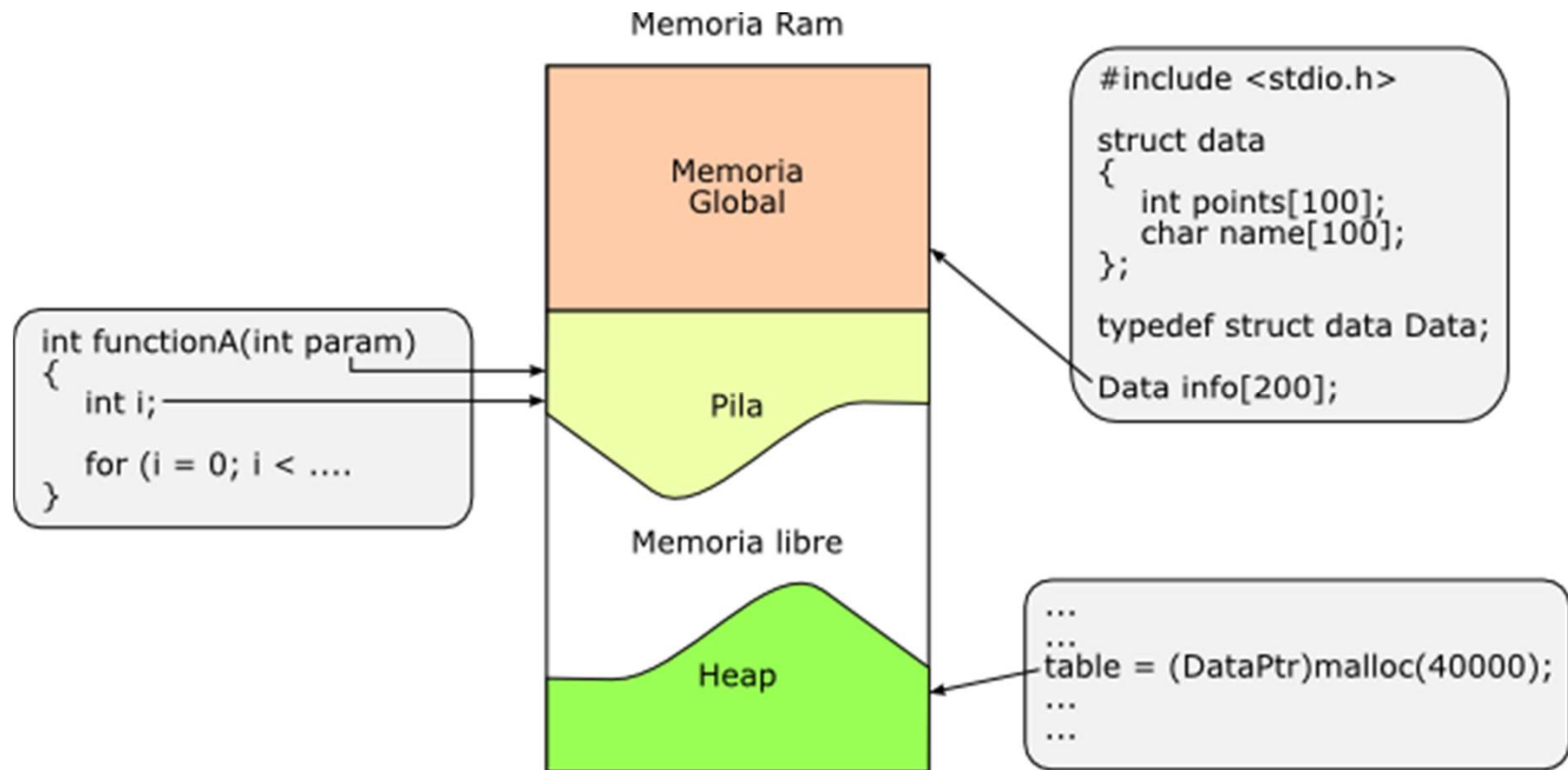


Manejo de memoria en tiempo de ejecución

Estructura de Datos y Algoritmos I
Profesora Elba Karen Sáenz García



Memoria a disposición del programador

- Memoria estática
- Heap
- stack

Funciones para el manejo de memoria dinámica

Prototipos funciones en stdlib.h

```
void *malloc(size_t size);
```

```
void *calloc(size_t nmemb, size_t size);
```

```
void free(void *ptr);
```

```
void *realloc(void *ptr, size_t size);
```

Un apuntador **void** direcciona datos de un tipo no especifico.

Uso malloc() y calloc()

Reservar memoria

```
int *vect1, n;  
printf("Número de elementos : ");  
scanf("%d", &n);  
vect1 = (int * )malloc(n *sizeof(int));  
  
if (vect1 == NULL) {  
    /* Error al intentar reservar memoria */ }  
free (vect1);
```

Uso malloc() y calloc()

Reservar memoria

```
int *vect1, n;
```

```
    printf("Número de elementos : "); scanf("%d", &n);
```

```
vect1 = (int *)calloc(n, sizeof(int));
```

```
if (vect1 == NULL) {
```

```
    /* Error al intentar reservar memoria */ }
```

- `free (vect1);`

Reutilizar el puntero

```
int *i;  
i = malloc(sizeof(int));  
  
free(i);  
i = NULL;
```

Ejemplo

- Reservar memoria en tiempo de ejecución para n enteros y n flotantes

Arreglos de estructura formados en tiempo de ejecución

```
struct Alumno{  
    char *nombre;  
    float promedio;  
    int edad; };
```

```
struct Alumno *p;
```

```
p=(struct Alumno *) malloc(sizeof(struct Alumno)*3);
```


Ejemplo 2

- Reservar espacio en tiempo de ejecución para un arreglo de estructuras Alumno. Cada elemento del arreglo almacenará el nombre, promedio y edad. Llenar y mostrar

```
#include<stdio.h>
#include<stdlib.h>
```

```
struct Alumno{
    char *nombre;
    float promedio;
    int edad; };
```

```
main(){  
    int i;  
    struct Alumno *p;  
    p=(struct Alumno *) malloc(sizeof(struct Alumno)*3);
```

```
for(i=0;i<3;i++){  
    (p+i)->nombre=(char *)malloc(30*sizeof(char));  
    printf("Introduce el nombre\n");  
    scanf("%s",p[i].nombre);  
  
    printf("\nIntroduce el promedio\n");  
    scanf("%f",&p[i].promedio);  
  
    printf("\nIntroduce edad\n");  
    scanf("%d",&p[i].edad);  
}
```

```
for(i=0;i<3;i++){  
    printf(" \nnombre %s", p[i].nombre);  
    printf("\nPromedio %f",p[i].promedio);  
    printf("\nEdad %d",p[i].edad);  
  
}  
  
free(p);  
}/*Fin main*/
```

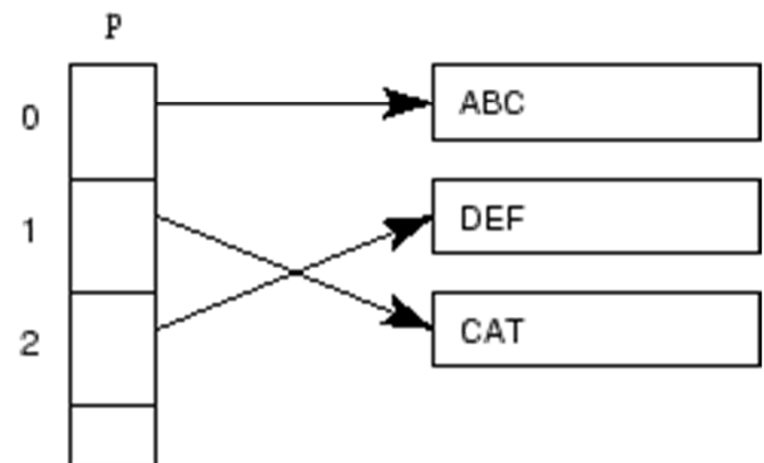
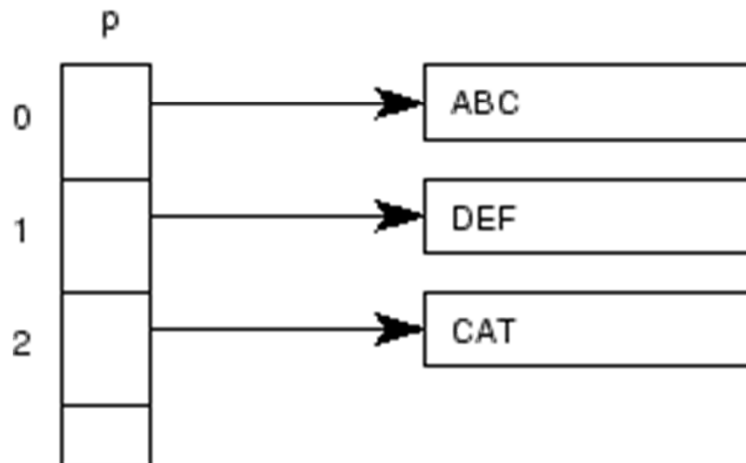
Arreglo de Apuntadores

- Los arreglos de apuntadores son una representación de datos que manejan de una forma eficiente y conveniente líneas de texto de longitud variable.

```
char *p[4];
```

ABC \nDEF \n CAT \n

p[0] p[1] p[2]



Ejemplo arreglo bidimensional con arreglo de apuntadores

```
int *m[5];  
/*arreglo de 5x3 */  
for(i=0;i<5;i++)  
    m[i]=(int *)malloc(3*sizeof(int));  
.....  
for(i=0;i<5;i++)  
    free(m);
```

Arreglo bidimensional apuntador a apuntador

```
int **m,nren=5,ncol=3;  
m=(int **)malloc(nren *sizeof(int *));  
    for(i=0;i<nren;i++)  
        m[i]=(int*)malloc(ncol*sizeof(int));  
  
.....  
for(i=0;i<nren;i++)  
    free(m[i]);  
free(m);
```


Ejemplo (En clase)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Prototipos de las funciones */
```

```
int **crear_matriz(int n, int m);
```

```
void destruir_matriz(int **matriz, int n, int m);
```

```
void llenar_matriz(int **matriz, int n, int m);
```

```
void liberar(int **matriz, int n, int m);
```

Relación arreglo bidimensional con doble apuntador

- `int M[3][5], i, j;`
- $M[i][j] \Leftrightarrow *(*(M+i)+j)$
- $\&M[i][j] \Leftrightarrow (*(M+i)+j)$

Entrada línea de comandos

```
main (int argc, char **argv) {  
    /* Este programa muestra los argumentos de la  
       línea de comandos */  
    int i;  
    printf("argc = %d\n\n",argc);  
    for (i=0; i<argc; ++i)  
        printf("\t\targv[%d]: %s\n", i, argv[i]);  
}
```

realloc()

`void *realloc(void *ptr, size_t size)`

- Resize the memory block pointed to by **ptr** that was previously allocated with a call to **malloc** or **calloc**.

Ejemplo

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main () {
    char *str;
    str = (char *) malloc(10);
    strcpy(str, "manejoDe");
    printf("Cadena = %s, direccion = %p\n", str, str);
    str = (char *) realloc(str, 1000000); strcat(str, ".com");
    printf("Cadena = %s, Direccion = %p\n", str, str); free(str);
    return(0);
}
```